Top 100 SQL Interview Q&A (Data Analytics) — 2025 | Part 2: Questions 26–50

> **Dialect locked: MySQL 8+ (Workbench).**\ **Format:** Difficulty · Comprehensive Question · Hint (+mini function recap) · **Unique Sample Table(s)** · Query · Steps · Final Answer · Easy Explanation.\ **Simplicity:** Stick to SELECT/WHERE/JOIN/GROUP BY/HAVING/ORDER BY, `CASE` pivots, and basic windows (ROW_NUMBER/RANK/DENSE_RANK/LAG). No exotic features.

---

26) Orders With NULL Delivery Date (Blinkit) — Difficulty: Easy

Comprehensive Question: List orders that haven't been delivered yet.

Hint: Use `IS NULL` to find missing delivery dates.\ **Function recap:** `IS NULL` checks for absent values.

Sample table — ``

order_id	customer_id	amount	delivery_date
1	C1	500	2025-07-02
2	C2	300	NULL
3	C3	250	NULL

Query (MySQL):

```
SELECT order_id, customer_id, amount
FROM orders_q26
WHERE delivery_date IS NULL;
```

Steps: Filter rows where `delivery_date` is NULL.\ **Final Answer:** Orders 2 and 3.\ **Easy Explanation:** No delivery date recorded ⇒ not delivered.

---

27) Count Orders by Status (Flipkart) — Difficulty: Easy

Comprehensive Question: How many orders are in each status?

Hint: Group by status and count.\ **Function recap:** `COUNT(*)` counts rows in each group.

Sample table — ``

order_id	status
10	placed
11	shipped
12	shipped
13	delivered

Query (MySQL):

```
SELECT status, COUNT(*) AS orders
FROM orders_q27
GROUP BY status
ORDER BY orders DESC;
```

Steps: Group → Count → Sort.\ **Final Answer:** shipped:2, placed:1, delivered:1.\ **Easy Explanation:** Tally per status bucket.

---

28) Average Order Value by City (Amazon India) — Difficulty: Easy

Comprehensive Question: Compute average order amount per city.

Hint: Join customers→orders and average by city.\ **Function recap:** `AVG()` calculates mean.

Sample tables — `**, **` `customers_q28`

customer_id	name	city
C1	Ana	Pune
C2	Ben	Mumbai

`orders_q28`

order_id	customer_id	amount
101	C1	300
102	C1	200
103	C2	500

Query (MySQL):

```
SELECT c.city, ROUND(AVG(o.amount),2) AS avg_order_value
FROM customers_q28 c
JOIN orders_q28 o ON o.customer_id=c.customer_id
```

```
GROUP BY c.city
ORDER BY avg_order_value DESC;
```

Steps: Join → average by city → order by avg.\ **Final Answer:** Mumbai:500; Pune:250.\ **Easy Explanation:** Typical spend per order in each city.

---

29) Percent of Total Sales by Category (Myntra) — Difficulty: Medium

Comprehensive Question: What percent of total sales does each category contribute?

Hint: Compute category totals and the overall total, then divide.\ **Function recap:** `CROSS JOIN` combines each row with the single total row.

Sample table — ``

category	amount
fashion	300
grocery	200
toys	100

Query (MySQL):

```
WITH cat AS (
  SELECT category, SUM(amount) AS cat_sales
  FROM sales_q29
  GROUP BY category
), tot AS (
  SELECT SUM(amount) AS total_sales FROM sales_q29
)
SELECT c.category,
       ROUND(100 * c.cat_sales / t.total_sales, 2) AS pct_of_total
FROM cat c CROSS JOIN tot t
ORDER BY pct_of_total DESC;
```

Steps: Category sums → total sum → percent per category.\ **Final Answer:** fashion 50%, grocery 33.33%, toys 16.67%.\ **Easy Explanation:** Slice of the pie by category.

---

30) Vendors With No Sales in Last 7 Days (Farmer's Market) — Difficulty: Medium

Comprehensive Question: Which vendors had no sales in the past 7 days?

Hint: Build recent-sellers set and anti-join against all vendors.\ **Function recap:** Anti-join via LEFT JOIN + `IS NULL`.

Sample tables — `**, **` `vendor_q30`

vendor_id	name
V1	Fresh Farm
V2	Green Veg
V3	Dairy Do

`purchases_q30`

market_date	vendor_id	amount
2025-07-01	V1	200
2025-07-08	V1	50
2025-07-08	V2	30

Query (MySQL):

```
SET @cutoff := DATE_SUB('2025-07-08', INTERVAL 7 DAY);
WITH recent AS (
  SELECT DISTINCT vendor_id
  FROM purchases_q30
  WHERE market_date >= @cutoff
)
SELECT v.vendor_id, v.name
FROM vendor_q30 v
LEFT JOIN recent r ON r.vendor_id=v.vendor_id
WHERE r.vendor_id IS NULL;
```

Steps: Define 7-day window → find recent sellers → left anti-join.\ **Final Answer:** V3 (no sales in window).\ **Easy Explanation:** Vendors inactive this week.

---

31) Most Popular Product per Category (Reliance Smart) — Difficulty: Medium

Comprehensive Question: Find the product(s) with highest units within each category.

Hint: Sum qty per product+category, rank by qty, keep rank 1.\ **Function recap:** `RANK()` ties share the same rank (1,1,3...).

Sample tables — `**, **` `product_q31`

product_id	product_category_id
P1	CAT1
P2	CAT1
P3	CAT2

`purchases_q31`

product_id	quantity
P1	10
P2	10
P3	7

Query (MySQL):

```
WITH q AS (
  SELECT p.product_category_id AS cat,
         cp.product_id,
         SUM(cp.quantity) AS qty
  FROM purchases_q31 cp
  JOIN product_q31 p ON p.product_id=cp.product_id
  GROUP BY p.product_category_id, cp.product_id
), r AS (
  SELECT cat, product_id, qty,
         RANK() OVER (PARTITION BY cat ORDER BY qty DESC) AS rnk
  FROM q
)
SELECT cat, product_id, qty
FROM r
WHERE rnk=1;
```

Steps: Sum → rank per category → rnk=1.\ **Final Answer:** CAT1: P1 & P2 (tie); CAT2: P3.\ **Easy Explanation:** Best-seller(s) per category.

---

32) Customers Who Bought From Multiple Vendors Same Day (Mandi) — Difficulty: Medium

Comprehensive Question: Which customers bought from 2+ vendors on the same day?

Hint: Count distinct vendors per customer-date; keep counts ≥2.\ **Function recap:** `COUNT(DISTINCT col)` counts unique values.

Sample table — ``

market_date	customer_id	vendor_id
2025-07-01	C1	V1
2025-07-01	C1	V2
2025-07-01	C2	V1

Query (MySQL):

```
SELECT market_date, customer_id,
       COUNT(DISTINCT vendor_id) AS vendors_that_day
FROM purchases_q32
GROUP BY market_date, customer_id
HAVING COUNT(DISTINCT vendor_id) >= 2
ORDER BY market_date, customer_id;
```

Steps: Group by date+customer → distinct vendor count → keep ≥2.\ **Final Answer:** C1 on 2025-07-01.\ **Easy Explanation:** Shopped at 2+ stalls that day.

---

33) Average Days Between Orders per Customer (Amazon) — Difficulty: Medium

Comprehensive Question: For each customer, what's the average gap in days between their orders?

Hint: Use previous order date and `DATEDIFF` to get gaps; average them.\ **Function recap:** `LAG()` looks at previous row; `DATEDIFF(a,b)` returns days.

Sample table — ``

customer_id	order_date
C1	2025-06-01
C1	2025-06-10
C1	2025-06-25
C2	2025-07-01
C2	2025-07-20

Query (MySQL):

```
WITH x AS (
  SELECT customer_id, order_date,
         LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)
AS prev_dt
  FROM orders_q33
```

```
)
SELECT customer_id,
       AVG(DATEDIFF(order_date, prev_dt)) AS avg_days_between
FROM x
WHERE prev_dt IS NOT NULL
GROUP BY customer_id;
```

Steps: Lag prev date → compute day gaps → average per customer.\ **Final Answer:** C1 ≈ 12 days; C2 ≈ 19 days.\ **Easy Explanation:** Typical waiting time between purchases.

---

34) Products Never Purchased (Flipkart) — Difficulty: Easy

Comprehensive Question: Which products have zero sales ever?

Hint: Left join products→order_items and keep rows with no match.\ **Function recap:** Anti-join via LEFT + `IS NULL`.

Sample tables — `**, **` `products_q34`

product_id	product_name
P1	Toy Car
P2	Puzzle
P3	Board Game

`order_items_q34`

product_id	qty
P1	2

Query (MySQL):

```
SELECT p.product_id, p.product_name
FROM products_q34 p
LEFT JOIN order_items_q34 oi ON oi.product_id=p.product_id
WHERE oi.product_id IS NULL;
```

Steps: Left join → keep NULL matches.\ **Final Answer:** P2, P3.\ **Easy Explanation:** Items no one bought.

---

35) Top 5 Customers by Spend (Nykaa) — Difficulty: Easy

Comprehensive Question: Who are the top 5 customers by total spend?

Hint: Sum `qty*price` per customer and sort.\ **Function recap:** Simple aggregation and ORDER BY.

Sample table — ``

customer_id	qty	price
C1	2	500
C2	1	800
C1	1	300
C3	3	200

Query (MySQL):

```
SELECT customer_id,
       ROUND(SUM(qty*price),2) AS total_spend
FROM order_items_q35
GROUP BY customer_id
ORDER BY total_spend DESC
LIMIT 5;
```

Steps: Sum spend per customer → sort desc → top 5.\ **Final Answer:** C2:800; C1:1300; C3:600 (ordered desc).\ **Easy Explanation:** Biggest spenders list.

---

36) Orders With Missing Customer (Data Quality) — Difficulty: Easy

Comprehensive Question: Find orders whose customer doesn't exist in the customers table.

Hint: Left join orders→customers and keep NULL on the right.\ **Function recap:** Orphan detection via left anti-join.

Sample tables — `**, **` `orders_q36`

order_id	customer_id
1	C1
2	C9

`customers_q36`

customer_id	name
C1	Ana

Query (MySQL):

```sql
SELECT o.order_id
FROM orders_q36 o
LEFT JOIN customers_q36 c ON c.customer_id=o.customer_id
WHERE c.customer_id IS NULL;
```

Steps: Left join → filter NULL customer.\ **Final Answer:** Order 2.\ **Easy Explanation:** Order refers to a non-existent customer.

---

37) Revenue by Week (Mandi) — Difficulty: Easy

Comprehensive Question: Sum weekly revenue grouped by Monday-start weeks.

Hint: Convert dates to week start using `DATE_SUB(date, INTERVAL WEEKDAY(date) DAY)`.\ **Function recap:** `WEEKDAY()` returns 0 (Mon) … 6 (Sun).

Sample table — ``

market_date	quantity	price
2025-07-01	2	100
2025-07-03	1	200
2025-07-09	3	50

Query (MySQL):

```sql
SELECT DATE_SUB(market_date, INTERVAL WEEKDAY(market_date) DAY) AS
week_start,
       SUM(quantity*price) AS revenue
FROM purchases_q37
GROUP BY week_start
ORDER BY week_start;
```

Steps: Map to week start → sum revenue → order by week.\ **Final Answer:** 2025-06-30 week: 400; 2025-07-07 week: 150.\ **Easy Explanation:** Totals per calendar week.

---

38) Repeat Buyers in a Month (Myntra) — Difficulty: Medium

Comprehensive Question: How many buyers placed more than one order in the same month?

Hint: First order per customer-month; count orders after that in same month.\ **Function recap:** `DATE_FORMAT(date,'%Y-%m-01')` gives month-start.

Sample table — ``

customer_id	order_date
C1	2025-07-01
C1	2025-07-12
C2	2025-07-03
C2	2025-08-01

Query (MySQL):

```
WITH m AS (
  SELECT customer_id,
         DATE_FORMAT(order_date,'%Y-%m-01') AS mth,
         order_date
  FROM orders_q38
), firsts AS (
  SELECT customer_id, mth, MIN(order_date) AS first_dt
  FROM m
  GROUP BY customer_id, mth
)
SELECT f.mth, COUNT(DISTINCT m.customer_id) AS repeat_buyers
FROM firsts f
JOIN m ON m.customer_id=f.customer_id AND m.mth=f.mth AND
m.order_date>f.first_dt
GROUP BY f.mth
ORDER BY f.mth;
```

Steps: Month bucket → first order per customer-month → count who ordered again.\ **Final Answer:** 2025-07-01: 1 repeat (C1).\ **Easy Explanation:** Customers who came back within the month.

---

39) SKUs With Price Increase vs Previous Month (Amazon) — Difficulty: Medium

Comprehensive Question: Which products' average selling price went up month-over-month?

Hint: Average price per product-month; self-join to prior month.\ **Function recap:** `DATE_SUB(month, INTERVAL 1 MONTH)` gets previous month.

Sample table — ``

product_id	order_date	price
P1	2025-06-10	10
P1	2025-07-01	12
P2	2025-07-02	5

Query (MySQL):

```
WITH mp AS (
  SELECT product_id,
         DATE_FORMAT(order_date,'%Y-%m-01') AS mth,
         AVG(price) AS avg_price
  FROM order_items_q39
  GROUP BY product_id, mth
)
SELECT curr.product_id, curr.mth, curr.avg_price, prev.avg_price AS prev_avg
FROM mp curr
JOIN mp prev
  ON prev.product_id=curr.product_id
 AND DATE_SUB(curr.mth, INTERVAL 1 MONTH)=prev.mth
WHERE curr.avg_price>prev.avg_price;
```

Steps: Avg price by product-month → join to previous → filter increases.\ **Final Answer:** P1 for July vs June.\ **Easy Explanation:** Price went up compared to last month.

---

40) Orders Outside Business Hours (Swiggy) — Difficulty: Easy

Comprehensive Question: List orders placed outside 09:00–21:00.

Hint: Use `HOUR(ts)` and filter by hour.

Sample table — ``

order_id	order_ts
1	2025-07-01 08:59:59
2	2025-07-01 10:00:00
3	2025-07-01 21:30:00

Query (MySQL):

```
SELECT order_id, order_ts
FROM orders_q40
WHERE HOUR(order_ts) < 9 OR HOUR(order_ts) > 21
ORDER BY order_ts;
```

Steps: Extract hour → filter outside range.\ **Final Answer:** Orders 1 and 3.\ **Easy Explanation:** Too early or too late.

---

41) Orders on Weekends (Dunzo) — Difficulty: Easy

Comprehensive Question: Return orders placed on Saturday or Sunday.

Hint: `DAYOFWEEK(date)` returns 1=Sun … 7=Sat.

Sample table — ``

order_id	order_date
501	2025-07-05
502	2025-07-06
503	2025-07-07

Query (MySQL):

```
SELECT order_id, order_date
FROM orders_q41
WHERE DAYOFWEEK(order_date) IN (1,7)
ORDER BY order_date;
```

Steps: Check weekday → keep 1 or 7.\ **Final Answer:** 501, 502.\ **Easy Explanation:** Weekend orders only.

---

42) Customers Who Never Ordered (Flipkart) — Difficulty: Easy

Comprehensive Question: Which customers have zero orders?

Hint: Left anti-join customers against orders.\ **Function recap:** Keep rows with NULL on the right.

Sample tables — `**, **` `customers_q42`

customer_id	name
C1	Ana
C2	Ben

`orders_q42`

order_id	customer_id
1	C1

Query (MySQL):

```sql
SELECT c.customer_id, c.name
FROM customers_q42 c
LEFT JOIN orders_q42 o ON o.customer_id=c.customer_id
WHERE o.customer_id IS NULL;
```

Steps: Left join → filter NULLs.\ **Final Answer:** C2.\ **Easy Explanation:** People who haven't bought yet.

---

43) Best Selling Day Overall (Amazon) — Difficulty: Easy

Comprehensive Question: Which day had the highest total sales?

Hint: Sum by date, sort desc, limit 1.

Sample table — ``

order_date	amount
2025-07-01	1000
2025-07-02	500
2025-07-02	400

Query (MySQL):

```sql
SELECT order_date, SUM(amount) AS revenue
FROM orders_q43
GROUP BY order_date
ORDER BY revenue DESC
LIMIT 1;
```

Steps: Group → sum → top 1.\ **Final Answer:** 2025-07-01 (1000) vs 2025-07-02 (900) ⇒ July 1.\ **Easy Explanation:** Day with the biggest revenue.

---

44) Top Category by Units (Reliance) — Difficulty: Easy

Comprehensive Question: Which category sold the most units?

Hint: Join items→products; sum qty by category; sort.

Sample tables — `**, **` `products_q44`

product_id	category
P1	Toys

product_id	category
P2	Games

`order_items_q44`

product_id	qty
P1	10
P2	7

Query (MySQL):

```
SELECT p.category, SUM(oi.qty) AS units
FROM order_items_q44 oi
JOIN products_q44 p ON p.product_id=oi.product_id
GROUP BY p.category
ORDER BY units DESC
LIMIT 1;
```

Steps: Join → sum per category → top.\ **Final Answer:** Toys (10 units).\ **Easy Explanation:** Which aisle moved most pieces.

---

45) Median (Small N) via Middle Value (Startup) — Difficulty: Medium

Comprehensive Question: Compute the median order amount for a small dataset.

Hint: Sort amounts and pick the middle row(s).\ **Function recap:** `ROW_NUMBER()` for positions; simple CASE to select middles.

Sample table — ``

order_id	amount
1	100
2	200
3	400
4	600

Query (MySQL):

```
WITH s AS (
  SELECT amount,
```

```
         ROW_NUMBER() OVER (ORDER BY amount) AS rn,
         COUNT(*) OVER () AS n
  FROM orders_q45
)
SELECT CASE WHEN n % 2 = 1
            THEN MAX(CASE WHEN rn = (n+1)/2 THEN amount END)
            ELSE ( MAX(CASE WHEN rn =  n/2     THEN amount END)
                 + MAX(CASE WHEN rn =  n/2 + 1 THEN amount END) )/2
       END AS median_val
FROM s;
```

Steps: Number rows → compute middle index(es) → pick/average values.\ **Final Answer:** Median = (200+400)/2 = 300.\ **Easy Explanation:** Middle value of the sorted list.

---

46) Month-to-Date vs Prior Month-to-Date (Nike) — Difficulty: Medium

Comprehensive Question: Compare current month-to-date sales with the same number of days last month.

Hint: Build two ranges using `DATE_FORMAT` and `DATE_SUB`, then sum.

Sample table — ``

order_date	amount
2025-07-01	100
2025-07-02	50
2025-06-01	80
2025-06-02	70

Query (MySQL):

```
SET @today := '2025-07-02';
SET @cm_start := DATE_FORMAT(@today,'%Y-%m-01');
SET @pm_same  := DATE_SUB(@cm_start, INTERVAL 1 MONTH);
SELECT 'MTD' AS label,
       (SELECT SUM(amount) FROM orders_q46
         WHERE order_date BETWEEN @cm_start AND @today) AS current_mtd,
       (SELECT SUM(amount) FROM orders_q46
         WHERE order_date BETWEEN @pm_same AND DATE_SUB(@pm_same, INTERVAL 1
DAY) + INTERVAL DAY(@today) DAY) AS prior_mtd;
```

Steps: Compute current range → compute prior range → compare sums.\ **Final Answer:** Current MTD=150; Prior MTD=150.\ **Easy Explanation:** This month vs same span last month.

47) Customers Who Ordered the Same Product Twice (Swiggy) — Difficulty: Medium

Comprehensive Question: Find customer+product pairs with 2+ purchases.

Hint: Group by both fields and filter by count ≥2.

Sample table — ``

customer_id	product_id
C1	P1
C1	P1
C2	P2

Query (MySQL):

```
SELECT customer_id, product_id, COUNT(*) AS times
FROM items_q47
GROUP BY customer_id, product_id
HAVING COUNT(*) >= 2;
```

Steps: Group → count → filter.\ **Final Answer:** (C1,P1).\ **Easy Explanation:** Same item bought more than once.

---

48) Orders With Negative Amount (QA) — Difficulty: Easy

Comprehensive Question: Identify orders with negative amounts.

Hint: Simple `WHERE amount < 0`.

Sample table — ``

order_id	amount
1	100
2	-50
3	0

Query (MySQL):

```sql
SELECT order_id, amount
FROM orders_q48
WHERE amount < 0;
```

Steps: Filter negatives.\ **Final Answer:** Order 2.\ **Easy Explanation:** Likely data error or refund.

---

49) Top City by Unique Buyers (Amazon) — Difficulty: Medium

Comprehensive Question: Which city has the most unique buyers?

Hint: Join customers→orders; count distinct customers per city.

Sample tables — `**, **` `customers_q49`

customer_id	city
C1	Pune
C2	Mumbai
C3	Pune

`orders_q49`

order_id	customer_id
1	C1
2	C3
3	C2
4	C1

Query (MySQL):

```sql
SELECT c.city, COUNT(DISTINCT o.customer_id) AS buyers
FROM customers_q49 c
JOIN orders_q49 o ON o.customer_id=c.customer_id
GROUP BY c.city
ORDER BY buyers DESC
LIMIT 1;
```

Steps: Join → distinct count per city → top 1.\ **Final Answer:** Pune (2 buyers).\ **Easy Explanation:** City with the widest buyer base.

---

50) Ratio: Returning / New Buyers by Month (Myntra) — Difficulty: Medium

Comprehensive Question: For each month, count new vs returning buyers.

Hint: First-ever purchase date per customer; compare buyer-months to that.

Sample table — ``

customer_id	order_date
C1	2025-06-01
C1	2025-07-02
C2	2025-07-05

Query (MySQL):

```
WITH firsts AS (
  SELECT customer_id, MIN(order_date) AS first_dt
  FROM orders_q50
  GROUP BY customer_id
), monthly AS (
  SELECT DATE_FORMAT(order_date,'%Y-%m-01') AS mth, customer_id
  FROM orders_q50
  GROUP BY mth, customer_id
)
SELECT m.mth,
       SUM(m.customer_id IN (
             SELECT customer_id FROM firsts
             WHERE DATE_FORMAT(first_dt,'%Y-%m-01')=m.mth
           )) AS new_buyers,
       SUM(m.customer_id IN (
             SELECT customer_id FROM firsts
             WHERE DATE_FORMAT(first_dt,'%Y-%m-01')<>m.mth
           )) AS returning_buyers
FROM monthly m
GROUP BY m.mth
ORDER BY m.mth;
```

Steps: First purchase per user → unique buyer-months → count new vs returning.\ **Final Answer:** 2025-06: new=1 ret=0; 2025-07: new=1 ret=1.\ **Easy Explanation:** First-timers vs repeaters by month.

> ✅ Q26–Q50 now each have **unique sample tables** (e.g., `orders_q26`, `customers_q28`, ..., `orders_q50`) and follow the same interview-ready format as Q1–Q25. Want me to proceed with **Q51–Q100** in the same style?